LAHORE UNIVERSITY OF MANAGEMENT SCIENCES
Department of Electrical Engineering

AI624 AI on Edge Devices
Fall 2026

Project Description

Total Marks: 100

Contribution to Final Assessment: 30%

Contents

1 Objectives **1**

2 Track A — Raspberry Pi 5 **3**
2.1 Project A1 — Long Context Tool-Calling Agent 4
2.2 Project A2 — Event-Triggered VLM Logging 9

3 Track B — Arduino Nano 33 BLE Sense **14**
3.1 Project B1— Teach-a-Gesture: On-Device Continual Learning Under 256 KB (Mandatory) 16
3.2 Project B2 — Adaptive-Bitrate Learned Compression for a Bandwidth-Constrained Camera Link (Option 1) 20
3.3 Project B3 — Cascaded Keyword Spotting for Real-Time Robot Control (Option 2) 24

4 Expectations and Scope of Work **29**
4.1 Scope of Work 29
4.2 Assessment 30

5 Timeline of Deliverables **31**

1 Objectives

The projects offered this year are organized primarily by the *target hardware platform*. Each platform imposes a different set of memory, compute, power, and I/O constraints, and the scope of every project has been narrowed or widened accordingly.

The tracks on offer are:

Track A: Projects on the Raspberry Pi 5

- **A1**: Long Context Tool-Calling Agent
- **A2**: Event-Triggered Vision-Language Model (VLM) Based Logging From A Video Stream

Groups choosing this track have an option to do **either one** of A1 or A2.

Track B: Projects on the Arduino Nano 33 BLE Sense

- **B1**: Teach-a-Gesture: On-Device Continual Learning Under 256 KB

- **B2**: Adaptive-Bitrate Learned Compression for a Bandwidth-Constrained Camera Link
- **B3**: Cascaded Keyword Spotting for Real-Time Robot Control

Groups choosing this track will do two projects. All groups choosing this track **have to do B1**. Additionally, they have an option to select **either one** of B2 or B3 as their second project.

The primary objectives of this course project are:

- Develop hands-on experience in deploying machine learning models on resource-constrained edge devices.
- Understand the challenges and optimization techniques for embedded AI systems, including memory, latency, and power constraints.
- Gain practical experience in model quantization, compression, and optimization for real-time edge inference.
- Learn to benchmark and compare edge AI systems using performance metrics such as inference latency, power consumption, and accuracy.
- Develop skills in end-to-end system integration, from data acquisition to real-time deployment and monitoring.
- Enhance technical documentation and presentation skills for complex edge computing solutions.

Students will work in teams of **four to five members**. All team members will receive the same score for the group project. However, the viva with the final presentation will be conducted in groups, and each student will be assessed and graded individually. Also note that any member of the group can be questioned or asked to present in the weekly milestone presentations preceding the final presentation.

Each track has a fixed capacity in terms of the number of groups it can accommodate; capacities are listed with each project description due to a limit on the hardware available. As such, the hardware will be assigned on a first-come, first-served basis.

2 Track A — Raspberry Pi 5

Lead Supervisor: Muhammad Ahmad
Maximum number of groups: 10

Platform Overview

The Raspberry Pi 5: a quad-core 64-bit Arm Cortex-A76 application processor at 2.4 GHz with 8 GB of LPDDR4X memory, running a full Linux stack with PyTorch, ONNX Runtime, and quantized transformer inference engines such as `llama.cpp`. The Cortex-A76 implements Armv8.2-A with NEON and the dot-product extension, but not the `i8mm` or SVE extensions, which determines which optimized integer kernels a runtime can dispatch to. Projects on this platform target the largest models in the course, small language and vision-language models of roughly 2B–8B parameters, inside complete end-to-end systems. For transformer workloads, prefill is bound by compute while decode is bound by memory bandwidth, since every generated token reads the full set of weights and the entire KV cache. With 8 GB shared between the operating system, the weights, the cache, and the rest of the pipeline, memory is as binding a constraint as speed.

Each group will be issued an 8 GB Raspberry Pi 5 for the duration of the project. Camera modules are not provided; projects that need a camera use a USB or network camera.

Approach

Every project starts from an application with concrete goals on latency, memory, or energy, and from an unoptimized baseline that you build yourself: an uncompressed (16-bit) model in the most direct pipeline. That baseline is expected to be far from usable, whether slow per request, heavy on memory, or costly in energy. The project is to close as much of that gap as possible through a sequence of measured changes (runtime tuning, compression, pipeline and preprocessing refinements, caching) and to explain each one. Projects do not impose fixed latency thresholds, since achievable throughput on this CPU is low; progress is judged against your own baseline. You will be assessed on the quality of the causal chain between a measured deficiency and the change that addresses it, as well as on the final system. A narrative of iterative, well-explained improvements is worth more than a single change that meets the goals.

For every change over the baseline, you are expected to report:

(a) The measurement on the previous version that motivated the change.

(b) What was modified, stated precisely enough for another group to reproduce.

(c) The delta on *every* reported metric, not only the one you targeted.

(d) Why the change produced that delta, argued from profiles, memory layout, or arithmetic.

Changes that made things worse are worth reporting and analyzing, provided they appeared reasonable; only improvements go into the final version.

The following apply to all Track A projects:

- **Fixed model family:** Each project fixes a model family at the start. Replacing the model with a newer or better family is not an improvement. Selection of size

within the family must be justified by your own measurements and must be such that there is a significant difficulty in running the unoptimized baseline. As such, a $\sim$ 1B model is not a good selection, but the near 8B selection makes for a good project.

- **Measurement:** Accuracy / end-task evaluation may run on a host with identical weights and pipeline; latency, memory, and power must be measured on the Pi. Report power in every project, alongside throughput, peak memory, and the project's task metrics.
- **Reproducibility:** Record runtime versions, your commits in a fork, build flags, model files, quantization formats, and all other settings so the build can be reproduced.

Evaluation

Evaluation rewards measured gains over your own baseline with task accuracy preserved, and a clear understanding of where each gain came from. The concrete grading breakdown is given in Section 4.2.

Minimum improvements. Achievable gains depend heavily on your baseline, meaning the model and pipeline you choose, so no fixed thresholds are set. Instead, we expect a well-developed improvement lineage with a significant number of improvements, each a concrete change measured with the full metric set and marked as kept or discarded; the required tasks of your project will account for most of them. Meeting this minimum earns full credit for results; falling below it earns partial credit, scaled by the gains achieved and by how well the shortfall is analyzed and explained. Breadth and rigor count for more than a single standout result: one large gain with incomplete requirements will score well below modest gains with complete requirements and thorough experimentation. Credit is given for:

- **Baseline and pipeline:** a correct, reproducible baseline and a trustworthy harness, which are prerequisites for every other claim.
- **Compression:** quantization and further compression steps, evaluated on both efficiency gains and preserved task metrics.
- **Report:** your methodology and each improvement; the ablations you ran; the effect of every change you tried; which changes worked and which were discarded, and why. Well-explained negative results earn credit.
- **Codebase and demo:** a clean, documented, reproducible codebase, and a live demo of the final stack on the Pi on inputs outside your evaluation set.

2.1 Project A1 — Long Context Tool-Calling Agent

Overview

Build an on-device agent that completes multi-step tasks by calling tools, with every token processed on the Pi. In each turn, the model emits a structured tool call, the tool runs locally, and its output is appended to the context, so sessions grow to several thousand tokens and the agent must still choose the right tool and arguments at the end.

Build an on-device agent that completes multi-step tasks by calling tools, with every token processed on the Pi. In each turn the model emits a structured tool call, the tool runs locally, and its output is appended to the context. The tools return large outputs (order and transaction histories, watch lists, message logs), so sessions grow to tens of thousands of tokens and the agent must still choose the right tool and arguments at the end.

The scope is **a single agent over a fixed set of $\sim$ 20-30 tools taken from the BFCL multi-turn backends in their long-context mode, text only, with sessions of 10k–20k tokens**. Multi-agent orchestration and models served from another machine are out of scope.

The system prompt and tool schemas must be prefilled before the first call, and prefill is compute-bound. The KV cache grows at $2 \cdot L \cdot H_{kv} \cdot d_h \cdot b$ bytes per token (layers, key/value heads, head dimension, bytes per element); for Qwen3-1.7B (L=28, H_{kv}=8, d_h=128) at 16 bits this is 112 KiB per token: 1.75 GiB at 16k and 3.5 GiB at 32k, on top of 3.8 GiB of BF16 weights. At 32k the BF16 weights and a 16-bit cache together no longer fit an 8 GB board beside the operating system and the rest of the pipeline, so reaching the full session length on the Pi requires compressing the weights, the cache, or both. Every decoded token reads all weights and the whole cache, so decode is bandwidth-bound and slows as the session grows; at 32k the cache is comparable in size to the weights. Finally, a client that re-sends the full history every turn pays prefill for the whole session each time, so total prefill grows quadratically with the number of turns.

Targets: Per-turn latency (tool output available $\rightarrow$ next call emitted), cold start to the first call, peak memory, and energy per session should all fall as far below your baseline as you can take them while session success holds. No fixed latency threshold is set, since achievable throughput on this CPU is low.

Starting Point

- **Runtime and model:** `llama.cpp` [1], served through `llama-server`; one open-weight instruct family with native tool calling, full (grouped-query) attention in every layer, and at least two sizes between roughly 0.5B and 8B, fixed for the project. Qwen3 [15] (0.6B, 1.7B, 4B, 8B) is recommended. Hybrid families whose layers are mostly linear-attention or recurrent (e.g. Qwen3.5) are not allowed: their KV cache is too small for the memory side of this project to matter.
- **Benchmark:** BFCL [2], restricted to 20–30 functions from its multi-turn backends, with the backends in long-context mode (as in BFCL's `multi_turn_long_context` category). The subset must include TradingBot: in long-context mode its ground-truth calls alone return about 11k tokens of tool output per task, against 2–4k for most other backends. Use per-turn accuracy for call accuracy, and multi-turn tasks using only those functions for session success.
- **Baseline.** Qwen3-1.7B (or the 2–4B member of your family) in BF16 with a 16k cache, which fits in 8 GB, with a stateless client re-sending the full history every turn, all schemas in the system prompt, unconstrained sampling, and default build settings. Note that `llama-server` reuses cached prompt prefixes by default; your baseline client must disable this.

Scope and Tasks

1. **Tool subset and long session:** Even in long-context mode, most BFCL sessions are shorter than this project targets, so add at least 5 tasks over the same backends that reach 16k–32k tokens through many calls and verbose outputs, each with a functional success check; at least two must need a detail from early in the session [3]. Hold out a test split.

2. **Client harness.** Logs per-turn tokens sent and reprocessed, prefill and decode time, tool time, and call validity (a well-formed call to an existing tool with schema-valid arguments).

3. **Baseline characterization:** Prefill and decode throughput at three or more context lengths; memory breakdown; verification of whether decode is bandwidth-bound (bytes read per token × tokens/s against measured bandwidth); per-turn and per-session wall-clock time; and the gap between single-turn accuracy and session success, with its cause tested by experiment.

4. **Runtime and serving (required):** First, compile for your hardware and set threads and pin cores; then keep the model loaded in a server that caches repeated prompts [4], so each turn processes only its new tokens. Check that the chat template renders earlier turns identically, since rewritten history can silently invalidate the cache. Optionally, constrain decoding with a grammar or JSON schema derived from the tool definitions, so every emitted call is well-formed, and measure its effect on call validity and decode speed. Speculative decoding [13] may also be added here; prompt-lookup drafting suits tool arguments that are copied from the context. `llama.cpp` supports EAGLE-3 natively.

5. **Quantization (required):** Weight quantization over at least two GGUF formats, comparing importance-matrix calibration on your own traces with generic text; and KV-cache quantization [5], checked on the early-detail tasks. You are allowed and expected to go beyond what basic `llama.cpp` provides here, including but not limited to mixed precision for weight quantization, and specialized KV cache quantization techniques (beyond simple Q4_0 or Q8_0). Report the effects of the quantization clearly, both in model response quality and performance (throughput, memory consumption).Optionally, include size selection within the family: compare a larger member at low precision against a smaller member at higher precision under the same memory budget.

6. **Further compression step (at least one required):** You are expected to further compress the model and the KV cache using techniques introduced in the course. These can include:

   - *Structured pruning* [6,7] with a light recovery fine-tune. The pruned model must still convert to GGUF and run in `llama.cpp`; supporting non-uniform widths, if needed, is part of the work.

   - *Distillation for tool calling*: LoRA fine-tuning [8] of the deployed member on tool-call traces generated on the host by a larger family member.

   - *Outlier suppression*: activation-aware scaling [9] or rotations [10,11], folded into the weights offline so the `llama.cpp` graph is unchanged, compared against plain quantization at the same format.

Report both call validity and session success; a method can preserve one and damage the other. Also report the benefit and trade-offs of your further compression clearly.

7. **Context management (at least one required):** Filter tool outputs before they enter the context, or retrieve only the relevant schemas per turn [12], summarize old turns, or bound the cache with attention sinks [14]. Retrieval shrinks the prefix but changes it every turn, working against caching; measure the trade-off. Summarization and bounded caches carry the same risk and should also be checked on the early-detail tasks. Optionally, add an operating-point analysis that sweeps the context budget against session success, latency, and energy.
8. **Integration:** A CLI or simple web chat (you can directly use `llama-server`) over the final stack, demonstrated live on a task outside the trace set, with turn latency decomposed and cold start measured.

Metrics for every change: session success, call validity, per-turn latency by component, cold and warm time to first call, peak memory, energy per session, and thermal state.

Expected Outcomes

- A working on-device agent with measured latency and energy gains over your baseline, the full per-change metric table with Pareto plots, and a clear account of each change or of why further gains were not achievable.
- A documented trace set, harness, and reproducible build; a live demo on the Pi.

Expected Milestones

1. **By first presentation (Week 1):** Tool subset, trace set, and harness; baseline characterized on the Pi.
2. **By second presentation (Week 2):** Prompt caching; weight and KV-cache quantization.
3. **By final presentation (Week 3):** Further compression and context management; integration and demo.

Tools and Measurement

Power: poll `vcgencmd pmic_read_adc` ($\sum V \times I$ over rails) from a separate process and integrate over steady-state windows; log `measure_temp` and `get_throttled` and flag throttled runs. Record the `llama.cpp` commit, build flags, and GGUF formats with every result.

Requirements

- **Baseline:** the BF16 2–4B configuration, run on the Pi through your harness on the held-out test split and fully characterized (Task 3) before any optimization.
- **Pipeline:** a reproducible build, the trace set with functional success checks, and a harness that logs every per-change metric; runtime tuning and prompt caching (Task 4) in place.

- **Compression:** weight and KV-cache quantization (Task 5), at least one further compression step (Task 6), and at least one context-management technique (Task 7).
- **Improvement lineage:** each change is applied on top of the last accepted configuration, forming one documented chain from baseline to final stack. Every step is measured with the full metric set on the same test split and marked as kept or discarded, with the reason. Discarded changes remain in the lineage.

References

[1] G. Gerganov *et al.*, `llama.cpp`. https://github.com/ggml-org/llama.cpp

[2] S. G. Patil *et al.*, "The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models," ICML 2025. Code: https://github.com/ShishirPatil/gorilla

[3] N. F. Liu *et al.*, "Lost in the Middle: How Language Models Use Long Contexts," TACL, 2024. arXiv:2307.03172.

[4] L. Zheng *et al.*, "SGLang: Efficient Execution of Structured Language Model Programs," NeurIPS 2024. arXiv:2312.07104.

[5] Z. Liu *et al.*, "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," ICML 2024. arXiv:2402.02750.

[6] X. Men *et al.*, "ShortGPT: Layers in Large Language Models are More Redundant Than You Expect," arXiv:2403.03853, 2024.

[7] S. Muralidharan *et al.*, "Compact Language Models via Pruning and Knowledge Distillation," NeurIPS 2024. arXiv:2407.14679.

[8] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," ICLR 2022. arXiv:2106.09685.

[9] J. Lin *et al.*, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," MLSys 2024. arXiv:2306.00978.

[10] S. Ashkboos *et al.*, "QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs," NeurIPS 2024. arXiv:2404.00456.

[11] Z. Liu *et al.*, "SpinQuant: LLM Quantization with Learned Rotations," ICLR 2025. arXiv:2405.16406.

[12] Y. Qin *et al.*, "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs," ICLR 2024. arXiv:2307.16789.

[13] Y. Leviathan *et al.*, "Fast Inference from Transformers via Speculative Decoding," ICML 2023. arXiv:2211.17192.

[14] G. Xiao *et al.*, "Efficient Streaming Language Models with Attention Sinks," ICLR 2024. arXiv:2309.17453.

2.2 Project A2 — Event-Triggered VLM Logging

Overview

Build an always-on camera system that notices when a new person enters a scene, describes them with a vision-language model (VLM) in a fixed schema, and posts the record to a log sink (dashboard, webhook, or broker). The VLM should run only when something new has happened.

The scope is **a static surveillance camera over a doorway that individuals pass through, mostly one at a time**, processed on an 8 GB Raspberry Pi 5. A person counts as "new" if they differ from the previously logged person: consecutive duplicates in the log are errors, while a person logged again after someone else is acceptable. Face recognition, naming anyone, crowd counting, and cloud-hosted models are out of scope.

The difficulty is a queue. Each description (encoder pass, prefill over image tokens, decoding the record) takes seconds on this CPU. If events reach the VLM at rate λ and it completes them at rate μ, then whenever $\lambda > \mu$ the backlog, and the time from entry to posted record, grows without bound; every design decision either raises μ or lowers the λ reaching the VLM. The pipeline need not see every frame: it samples the stream at a rate you choose (e.g. 2–5 fps instead of 25–30), trading load against the risk of missing brief entries.

Targets. Time from entry to posted record should be as short as you can make it and must stay bounded (a backlog that keeps growing is a failure); at most 2 duplicate and 2 false records per hour of footage. No fixed latency threshold is set.

Starting Point

- **Camera:** No Pi camera is issued: stream from a USB webcam, or a phone or IP camera over the network (RTSP or MJPEG). Transport is part of the latency decomposition.
- **Platform:** OpenCV or FFmpeg for capture, `llama.cpp` [1] multimodal support for the VLM, and ONNX Runtime, ncnn, or TensorFlow Lite for any detector.
- **Model family:** Qwen3.5 [2] (0.8B, 2B, 4B, 9B; natively multimodal) is recommended, fixed for the project; the deployed size is yours to determine by measurement. Its vision encoder works at the input's native resolution, so the number of image tokens, and the encoder's cost, scale with the pixels you send (an 800×600 frame is about 475 image tokens). Before relying on any per-frame figure, check how the runtime preprocesses images: some families upscale every frame into a fixed grid of tiles.
- **Baseline:** The 2B member with its language decoder in BF16 and its vision encoder in F16, at the chat template's defaults (including thinking), given the full frame at a fixed interval (e.g. every 30 s), prompted for a JSON record of every visible person, posting every record, with no detector, tracking, caching, or bound on the queue.

Scope and Tasks

1. **Dataset:** The ChokePoint dataset [3] is recommended: surveillance video (800×600, 30 fps) from three cameras mounted above two portals, through which

54 subjects walk, entering or leaving, mostly one at a time. The two portals were recorded a month apart, so clothing changes between them, and two sequences are crowded with continuous occlusion. Only the frames around each passage are released (plus 100 empty background frames per sequence), and the ground truth marks only the subject id and eye positions of the person nearest the camera; some passages contain a second person, so check every passage you use by eye. Compose test streams by splicing passages with stretches of empty scene between them, controlling the entry rate and how often the same subject passes twice in a row (which must not produce a second record). ChokePoint has no attribute labels: label 4–6 attributes once per subject and portal (e.g. upper and lower clothing colour, outerwear, carrying a bag or object, glasses), following the schemas of [4,5]. Insert hard negatives into the gaps (lighting changes, shadows, motion without a person). Split by subject and hold out a test split; the demo streams come from it. Use the crowded sequences as a separate stress test and report entry recall and duplicates on them separately. Optionally, add low-light streams (e.g. darkened or gamma-reduced clips) and report robustness on them separately.

2. **Replay harness:** Serves recorded video through the same interface as the live camera, paces frames by wall clock at the sampled rate without waiting for the pipeline (dropping and logging frames that arrive while busy), and timestamps frames and records. To keep evaluation manageable, it may replay footage faster than real time (e.g. 2–4×); this raises the effective arrival rate, so state the speed-up with every result and report time-to-post and dropped frames at 1×. The harness scores entry recall, duplicate and false records, time-to-post, attribute accuracy, and hallucination rate (attributes asserted that the annotation marks as absent).

3. **Baseline characterization:** Per-record VLM cost (image tokens, encoder, prefill, decode) against resolution; service rate and the highest stable sampling rate; non-model path cost; memory; and baseline failures, with causes identified by experiment.

4. **Runtime and scheduling (required):** A persistent VLM process with the prompt cached; capture decoupled from description by a bounded queue with an explicit drop policy (report queue length over time); core affinity between detector and VLM.

5. **Gating (required):** Invoke the VLM at most once per new person. A typical cascade is a change gate [6], a person detector, and a simple tracker [7]. Report the operating curve of your main threshold against VLM invocations, missed entries, and power. Optionally, select the best frame within a track (e.g. by sharpness, size, or detector confidence) before invoking the VLM, or add re-identification [12] so a returning person is matched to their existing record; report how either changes attribute accuracy and the record count.

6. **Input reduction (required):** Crop to the detected person, and sweep resolution as a Pareto curve of image tokens against attribute accuracy. Optionally, prune or merge visual tokens [13,14] and report them on the same Pareto curve.

7. **Quantization (required):** The VLM's language decoder over at least two of `llama.cpp`'s GGUF weight formats; the vision encoder below F16, with its sensitivity measured separately from the decoder's (on CPU the encoder is the largest single cost of a description); and the detector to INT8. You are allowed and expected to

go beyond what basic `llama.cpp` provides, for example with mixed precision across layers. Report the effects of quantization clearly, both on record quality (attribute accuracy, hallucination rate) and on performance (throughput, memory).

8. **Further compression step (at least one required):** You are expected to further compress the VLM using techniques introduced in the course. These can include:

   - *Outlier suppression*: activation-aware scaling [8] or rotations [9] applied to the language decoder before quantization, folded into the weights offline so the `llama.cpp` graph is unchanged, and compared against plain quantization at the same format.
   - *Structured pruning* of the language decoder [10] with a recovery fine-tune; the pruned model must still convert to GGUF and run in `llama.cpp`.
   - *Fine-tuning* with LoRA [11] on the record schema, optionally on records generated on the host by a larger family member, merging the adapter before quantization.

   Report attribute accuracy and hallucination rate separately; a method can preserve one and damage the other. Once one technique is complete, a second (e.g. distillation from a larger family member) may be added. Report the benefit and trade-offs of each clearly.

9. **Integration:** Held-out streams served through the camera interface, records shown as they arrive, entry-to-post latency decomposed by stage. A short live-camera segment is encouraged, with consent of anyone filmed.

Metrics for every change: entry recall, duplicate and false-record rates, time-to-post, attribute accuracy, hallucination rate, VLM duty cycle (fraction of time the VLM is busy), dropped frames, peak memory, power, energy per record, and thermal state.

Fallbacks

If VLM output does not parse reliably, constrain decoding to the schema first. If the network camera is unreliable, use a USB webcam or replayed footage only for the demo. If the pipeline cannot keep up, lower the sampling rate and report the cost in missed entries.

Expected Outcomes

- A working logging system with measured gains in time-to-post, VLM duty cycle, and energy per record over your baseline, the per-change metric table, Pareto plots, and gating operating points.
- The documented stream composer, annotations, harness, and reproducible build (including camera setup); a live demo on the Pi.

Expected Milestones

1. **By first presentation (Week 1):** Dataset, camera link, and replay harness; baseline characterized on the Pi.

2. **By second presentation (Week 2):** Runtime and scheduling, gating with its operating curve, input reduction.

3. **By final presentation (Week 3):** Quantization and further compression; integration and demo.

Tools and Measurement

Power: poll `vcgencmd pmic_read_adc` ($\sum V \times I$ over rails) from a separate process and integrate over steady-state windows; log `measure_temp` and `get_throttled` and flag throttled runs. Record the `llama.cpp` commit, detector runtime, build flags, and quantization formats with every result.

Requirements

- **Baseline:** the BF16 full-frame, fixed-interval configuration from the Starting Point, run on the Pi through the replay harness on the held-out test streams and fully characterized (Task 3) before any optimization.
- **Pipeline:** a reproducible build including camera setup, the composed test streams with annotations, and a replay harness that scores every per-change metric; runtime and scheduling (Task 4) in place.
- **Load reduction:** gating with its operating curve (Task 5) and input reduction with its resolution Pareto curve (Task 6).
- **Compression:** quantization of the language decoder and detector (Task 7), and at least one further compression step (Task 8).
- **Improvement lineage:** each change is applied on top of the last accepted configuration, forming one documented chain from baseline to final stack. Every step is measured with the full metric set on the same test streams and marked as kept or discarded, with the reason. Discarded changes remain in the lineage.

References

[1] G. Gerganov *et al.*, `llama.cpp`. https://github.com/ggml-org/llama.cpp

[2] Qwen Team, Qwen3.5 model family. https://huggingface.co/Qwen/Qwen3.5-2B

[3] Y. Wong, S. Chen, S. Mau, C. Sanderson and B. C. Lovell, "Patch-based Probabilistic Image Quality Assessment for Face Selection and Improved Video-based Face Recognition," CVPR Workshops 2011. Data: https://arma.sourceforge.net/chokepoint/

[4] X. Liu *et al.*, "HydraPlus-Net: Attentive Deep Features for Pedestrian Analysis," ICCV 2017. arXiv:1709.09930.

[5] Y. Lin *et al.*, "Improving Person Re-identification by Attribute and Identity Learning," Pattern Recognition, 2019. arXiv:1703.07220.

[6] Z. Zivkovic, "Improved Adaptive Gaussian Mixture Model for Background Subtraction," ICPR 2004.

[7] Y. Zhang *et al.*, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," ECCV 2022. arXiv:2110.06864.

[8] J. Lin *et al.*, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," MLSys 2024. arXiv:2306.00978.

[9] S. Ashkboos *et al.*, "QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs," NeurIPS 2024.

arXiv:2404.00456.

[10] X. Men *et al.*, "ShortGPT: Layers in Large Language Models are More Redundant Than You Expect," arXiv:2403.03853, 2024.

[11] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," ICLR 2022. arXiv:2106.09685.

[12] K. Zhou *et al.*, "Omni-Scale Feature Learning for Person Re-Identification," ICCV 2019. arXiv:1905.00953.

[13] D. Bolya *et al.*, "Token Merging: Your ViT but Faster," ICLR 2023. arXiv:2210.09461.

[14] L. Chen *et al.*, "An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models," ECCV 2024. arXiv:2403.06764.

3 Track B — Arduino Nano 33 BLE Sense

Maximum number of groups: 12

Platform Overview

The Arduino Nano 33 BLE Sense is a resource-constrained microcontroller platform: a 64 MHz Arm Cortex-M4F (nRF52840) with roughly 1 MB of flash and 256 KB of SRAM, no operating system, and little compute to spare. It does, however, carry a rich on-board sensor suite (IMU, digital microphone, proximity/light/gesture, and temperature, humidity and pressure sensors) together with a Bluetooth Low Energy radio. This makes it the natural platform for *TinyML*: keyword spotting, gesture and activity recognition, learned compression of sensor data, and on-device training. Deployment typically goes through TensorFlow Lite for Microcontrollers with CMSIS-NN kernels, or another bare-metal runtime where a project specifies one. The binding constraints are SRAM working memory, flash footprint, and the time available per input, so every project on this track has to show that its system fits and keeps up, not only that it is accurate.

Approach

Every project starts from an application with concrete goals and from an unoptimized baseline: either a deliberately plain model you build yourself or a published reference model you reproduce. That baseline is expected to fall short, whether it misses the real-time budget, fails under realistic input, or does not fit in memory at all. The project is to identify those deficiencies by measurement and close as much of the gap as possible through a sequence of measured changes (architectural refinements, data and pipeline improvements, compression, and runtime tuning), explaining each one. Progress is judged against your own baseline. You will be assessed on the quality of the causal chain between a measured deficiency and the change that addresses it, as well as on the final system. A narrative of iterative, well-explained improvements is worth more than a single change that beats them.

For every change over the baseline, you are expected to report:

(a) The specific measurement on the previous version that motivated the change. Quantitative as well as qualitative reasons qualify.

(b) What was modified, stated precisely enough for another group to reproduce.

(c) The delta on *every* metric in the reporting table, not only the one you targeted. A latency win paid for in flash is still a trade, and you must establish both the cost and the gain.

(d) Why the change produced that delta, argued from the operator profile, the memory layout, or the arithmetic.

Changes that made things worse are worth reporting and analyzing, provided they appeared reasonable; only improvements go into the final version.

The following apply to all Track B projects:

- **Measurement:** Accuracy sweeps may run on a CPU/GPU host with the identical model, quantization, and preprocessing, provided a device spot-check confirms host–

device agreement. Latency, peak SRAM, and flash must be measured on the device. Power and energy are measured only where the project specifies.

- **Reproducibility:** The Arduino core, runtime and library versions (e.g. TFLM, CMSIS-NN, or the project's training framework), your commits, build flags, arena size, quantization settings, and board revision must be recorded so the build can be reproduced.

Evaluation

Evaluation rewards measured gains over your own baseline with task accuracy preserved, and a clear understanding of where each gain came from. The concrete grading breakdown is given in Section 4.2.

Scope of improvements. No fixed number of steps is set, since the projects on this track differ in objectives and the required tasks already define most of the work. We do expect a documented improvement lineage that covers every required task of your project, with each step measured on the full metric set and marked as kept or discarded. Completing the required tasks with this lineage earns full credit for results; falling short earns partial credit, scaled by the gains achieved and by how well the shortfall is analyzed and explained. Breadth and rigor count for more than a single standout result: one large gain with incomplete requirements will score well below modest gains with complete requirements and thorough experimentation. Credit is given for:

- **Baseline and pipeline:** a correct, reproducible baseline and a trustworthy evaluation path, which are prerequisites for every other claim.
- **Compression:** the compression techniques your project requires, evaluated on both efficiency gains and preserved task metrics.
- **Report:** your methodology and each improvement; the ablations you ran; the effect of every change you tried; which changes worked and which were discarded, and why. Well-explained negative results earn credit.
- **Codebase and demo:** a clean, documented, reproducible codebase, and a live demo of the final system on the board on inputs outside your evaluation set.

3.1 Project B1— Teach-a-Gesture: On-Device Continual Learning Under 256 KB (Mandatory)

Lead Supervisors: Usman Ahad and Wasif Ahmad

Overview

Build a motion-gesture recognizer on the Arduino Nano 33 BLE Sense that users can teach new gestures after deployment, one at a time with about five examples each, using the board's own IMU. The board must learn each new gesture on the device and still recognize every gesture it knew before.

The scope is **class-incremental learning of isolated gestures, performed entirely on the microcontroller**. Gestures are segmented from the IMU stream by a motion threshold. Capture, segmentation, forward and backward passes, parameter updates, and replay all run on the Nano; the host may only send the label of the gesture being taught and display results.

The central problem is **catastrophic forgetting**: training on a new gesture alone overwrites what the network knew, and accuracy on old gestures collapses. The standard remedy is *replay* [4], mixing stored old examples into every training step. *Latent replay* [2] freezes the network up to a split layer k and stores old examples as their layer-k activations, so only the layers after the split are trained. The closest published system [3] needs up to 64 MB for this; here, learning must fit in whatever part of the Nano's 256 KB of SRAM is left after inference.

SRAM is the only memory available for learning. The replay buffer, saved activations, gradients, trainable weights, and optimizer state must all fit in the SRAM left after the core, drivers, and inference. Flash holds code and frozen weights and must not be written at run time. Every trainable float32 parameter costs 12 bytes with SGD and momentum, paid for in replay samples: a deeper split leaves more room to remember but less of the network able to learn; where that balance falls, and how quantizing the buffer moves it, is what this project measures.

Starting Point

You are given the task, data, and protocol; you build the first model yourself.

- **Platform.** Arduino Nano 33 BLE Sense and its on-board IMU; record the board revision, as the IMU differs between them.
- **Data.** 6DMG [1]: 20 motion gestures recorded with a Wii Remote's accelerometer and gyroscope at 60 Hz; use the MATLAB export and the 21 right-handed users. The 14 straight-line gestures (swipes, pokes, *V*, *X*) are pre-trained. The 6 rotational gestures are taught on the device in a fixed order: the four circles, then *twist clockwise* (a sharp $\sim 90^\circ$ turn and back) and its mirror, *twist anticlockwise*.
- **Protocol.** A fixed user-disjoint split of 14 pre-training users and 7 *visitors*. For each new gesture, take 5 examples from one visitor and train on the device; after all six, test on every remaining visitor recording over all 20 gestures. Repeat per visitor and report mean and spread. The headline metric is *balanced accuracy*, the mean of old-gesture and taught-gesture accuracy; differences within the spread between visitors are ties.

- **Training framework.** Any framework that performs backpropagation on the microcontroller, such as AIfES [7]; a hand-written backward pass is not expected.
- **Evaluation path.** Sweeps may run on the host with the identical model, quantization, buffer, and update rule, provided a device spot-check confirms host–device agreement. Memory and timing must be measured on the device.

Scope and Tasks

1. **Baseline and learning budget.** Train a deliberately plain 1D CNN on the base gestures (strided 1D convolutions, global pooling, one dense layer; no depthwise-separable blocks, recurrence, attention, or architecture search). Quantize to INT8, deploy, and report host and device accuracy on visitors, per-layer parameters and MACs, latency per stage, flash, and an SRAM breakdown. The SRAM that remains is your *learning budget*; hold it fixed for the main comparison. Map the Nano's IMU to the 6DMG convention (axes, units, rate), check it on a few live base gestures, and report segmentation misses and false triggers.
2. **Establish forgetting and the memory wall.** Teach the new gestures on the device without replay, training the head alone and then deeper layers, and report old and taught accuracy after each gesture. Derive the SRAM needed to train the full network, compare it with the measured high-water mark, and identify which term exceeds the budget first.
3. **Replay under the budget (required).** Compare raw replay with latent replay at three or more split points, with the buffer at each split sized to whatever the budget leaves after training memory. Test at least two policies for drawing replay samples, and justify your chosen split from the measurements.
4. **Compression (required, both).**
   - *Replay quantization*: store latents at 8 bits and at least one lower bit-width [3], reporting samples gained, accuracy cost, and whether a previously starved split becomes usable.
   - *One further technique*: structured pruning or width reduction with QAT, distillation from a host-side teacher, or low-rank factorization of the trainable layers. Report its inference benefit and what the freed SRAM buys. Include a comparison of a narrower network trained in full against selective training of your wider one at the same budget.
5. **Integration.** A live teach-and-recognize loop: the host sends the label of a new gesture, which puts the board into teaching mode; the user performs it five times, the board trains, and recognition resumes over all known gestures, with results shown on the host. Report teaching time per gesture and end-of-gesture-to-label latency by stage.

Metrics for every change: balanced, old, and taught accuracy (final and per taught gesture), twist mirror confusion, training time per gesture, latency per stage, peak SRAM by component, replay capacity, and flash.

Fallbacks

If convolutions cannot be trained on the device, train dense layers on pooled latents. If the Wii Remote-to-Nano gap cannot be closed, stream held-out visitor recordings to the board over serial for the demo. If threshold segmentation is unreliable, mark gesture start and end from the host over serial. Report every substitution.

Expected Outcomes

- A recognizer that learns new gestures on the device without forgetting, improving measurably on naive on-device training, with a clear account of each change.
- The reporting table, balanced accuracy against split and bit-width at a fixed budget, and SRAM accounting checked against measurement.
- A reproducible build and a live demo in which a visitor teaches the twists and the base gestures still work.

Expected Milestones

Week 1

- Prepare the 6DMG data using 14 users for pre-training and 7 users for evaluating newly taught gestures.
- Train the baseline CNN on the 14 base gestures, quantize it to INT8, and deploy it on the Nano.
- Match the Nano's IMU readings to the dataset and test gesture segmentation, reporting missed gestures and false triggers.
- Measure baseline accuracy, per-layer parameters and MACs, latency, flash, and SRAM. Fix the remaining SRAM as the learning budget.
- Teach the six new gestures without replay using five examples each. Compare head-only and deeper-layer training, reporting old and new gesture accuracy after each addition.
- Calculate full-network training memory and use device measurements to identify what exceeds the learning budget.

Week 2

- Compare raw replay with latent replay at three or more split points under the same learning budget.
- Compare at least two replay-sampling policies and justify the selected split and policy.
- Quantize stored latents to 8 bits and at least one lower bit-width. Report replay capacity and accuracy at each setting.
- Repeat the teaching experiment for each visitor and report accuracy after every new gesture, final balanced accuracy, variation across visitors, and twist mirror confusion.

- Measure training time, latency, SRAM, replay capacity, and flash for each configuration. Verify host–device agreement if evaluation sweeps run on the host.

Week 3

- Apply one further compression technique and report its inference benefit and how the freed SRAM improves learning or replay.
- Compare full training of a narrower network with selective training of the wider network at the same learning budget.
- Complete the live teaching loop with gesture capture, training, replay, and recognition running on the Nano.
- Demonstrate teaching the twists with five examples each while retaining recognition of the base gestures.
- Complete the final evaluation and reporting table, including teaching time and recognition latency by stage.
- Prepare the reproducible code and document all changes, results, and any fallback substitutions.

Tools and Measurement

Time stages with the DWT cycle counter or `micros()`, and report peak SRAM as a measured run-time high-water mark. Power is not measured. Record core, library and framework versions and board revision with every result.

References

[1] M. Chen, G. AlRegib and B.-H. Juang, "6DMG: A New 6D Motion Gesture Database," MMSys 2012. https://github.com/olivesgatech/6DMG

[2] L. Pellegrini *et al.*, "Latent Replay for Real-Time Continual Learning," IROS 2020. arXiv:1912.01100.

[3] L. Ravaglia *et al.*, "A TinyML Platform for On-Device Continual Learning with Quantized Latent Replays," IEEE JETCAS, 2021. arXiv:2110.10486.

[4] D. Rolnick *et al.*, "Experience Replay for Continual Learning," NeurIPS 2019. arXiv:1811.11682.

[5] J. Lin *et al.*, "On-Device Training Under 256KB Memory," NeurIPS 2022. arXiv:2206.15472.

[6] H. Cai *et al.*, "TinyTL: Reduce Activations, Not Trainable Parameters for Efficient On-Device Learning," NeurIPS 2020. arXiv:2007.11622.

[7] Fraunhofer IMS, "AIfES for Arduino." https://github.com/Fraunhofer-IMS/AIfES_for_Arduino

3.2 Project B2 — Adaptive-Bitrate Learned Compression for a Bandwidth-Constrained Camera Link (Option 1)

Lead Supervisor: Umer Irfan

Overview

Build a learned image compression system that actually runs on the Arduino Nano 33 BLE Sense:

$$\text{camera} \rightarrow \text{MCU encoder} \rightarrow \text{latent} \rightarrow \text{BLE} \rightarrow \text{host decoder} \rightarrow \text{image}.$$

The encoder is as large as the complete on-device pipeline allows. The question is how much compression quality and progressive bitrate control that encoder can deliver, and how that scales with the decoder.

MCUCoder [1] is the starting idea: a very small encoder on the device, most of the model on the receiver, and latent channels ordered by importance. If bandwidth drops, the sender does not re-encode. It sends fewer channels. That property is mandatory here. The prefixes $z_{1:1}, z_{1:2}, \ldots, z_{1:N}$ must be progressively better representations of the same frame, with $N \geq 10$ independently droppable channels or units. There is no maximum N and no fixed latent width, spatial size or channel size. Do not treat 12 channels as a magic number because MCUCoder used it. Decide whether many small channels or fewer large ones works better under this hardware, and justify it.

The receiver is not resource constrained. A mandatory part of the project is to find out how much a larger receiver-side decoder can compensate for a weak MCU encoder, first by varying CNN decoder capacity and then, separately, by changing the decoder architecture. Keep in mind that KODAK is your evaluation dataset. Do not train on KODAK, For training you can use any dataset.

Setup

- **Hardware.** Arduino Nano 33 BLE Sense with the camera from the Arduino Tiny Machine Learning Kit. The encoder runs on the Nano. The decoder runs on a laptop or phone.
- **Reference.** Read MCUCoder [1] first. Understand why the encoder is small, why the decoder can be large, how the channels become ordered, and how sending a prefix gives multiple bitrates.
- **Data.** Train on a public image dataset of your choice. Evaluate on Kodak.
- **BLE.** Use the payload throughput you actually measure, not the nominal PHY rate.

Metrics

These two sets are referred to throughout.

Quality metrics: MS-SSIM, LPIPS (Using AlexNet Backbone, Use [4]), PSNR, MSE, payload bits per pixel and payload bytes per frame, reported over multiple values of k

and not only for the full latent. BLE headers and protocol overhead do not count toward bpp.

Device metrics: encoder parameters, encoder MACs, on-device encoder latency, peak SRAM and flash usage.

Report both sets for every meaningful model change. For the integrated system also report measured BLE payload throughput, achieved frame rate, and capture-to-display latency split into capture, encoding, transmission and reconstruction/display. Host decoder latency, host memory and power are not targets.

Tasks

1. **Baseline codec.** Before touching the microcontroller, train a lightweight CNN encoder and CNN decoder in the style of MCUCoder, with a training objective of your choice. Verify that reconstruction improves as k increases. If the model only works with all channels present, you have not implemented the central idea.

2. **JPEG comparison.** Build a JPEG rate–distortion curve over multiple quality settings on the same test images. Beating JPEG is not required. The comparison is.

3. **Find the memory wall.** Try to deploy the initial encoder and measure what happens. Break down memory into model/runtime, camera buffers, application buffers, BLE and anything else that matters. Do not shrink the model because "it probably will not fit." Establish why it does not fit first. The only hard target is that the complete camera–encoder–BLE pipeline runs on the actual device. Inference working in isolation does not count.

4. **Explore the encoder.** You are not restricted to a fixed list of techniques. Useful directions include input resolution and representation, width and depth, downsampling, depthwise-separable convolutions, bottleneck size, latent spatial resolution, number of channels and information per channel, pruning and factorization. The goal is the strongest encoder that runs inside the full camera–encoder–BLE pipeline. Once you have it, fix its architecture and latent shape. There is no separate encoder-capacity sweep.

5. **Try one different encoder idea.** Explore at least one idea that changes how the representation is learned, beyond resizing or uniform width reduction: lightweight attention, transformer-style blocks, learned tokenization, VAE-style latents, alternative bottlenecks or another modern learned-compression idea. It does not have to win. A sensible idea that fails with a correct explanation is still useful.

6. **CNN decoder-capacity ablation.** Keep the encoder architecture, latent shape and payload fixed. Build at least four CNN decoders $D_1, \ldots, D_4$ of increasing capacity in the same architectural family, and train the encoder jointly with each one, so that only decoder capacity changes. Evaluate each pair at $k = 1, 3, 6, \min(12, N), N$. This should answer how much quality a larger decoder recovers from the same constrained encoder and payload. Pay particular attention to small k.

7. **Train the final configuration.** The sweep is an ablation, not the final model. Choose the most useful encoder–decoder pair from it and train it properly as the main codec.

8. **Alternative decoder architecture.** With the final encoder, latent and payload fixed, replace the CNN decoder with at least one substantially different model, such as a transformer, a generative or diffusion/refinement model, or a different variational architecture. It may be far larger than anything that would fit on the Nano. Keep this separate from the capacity ablation: that one asks whether more capacity helps, this one asks whether a better architecture extracts more from the same latent.

9. **QAT.** Only after the encoder architecture is locked, perform quantization-aware training and compare the model immediately before and after. Choose the latent precision yourself and justify it. It does not have to be MCUCoder's 5 bits.

10. **Bitrate control.** Treat your measured maximum sustained BLE payload throughput as the upper budget, then impose smaller budgets artificially. For each budget, send the largest latent prefix that fits. No link controller or packet-loss simulator is required. Report quality against transmitted payload and achieved frame rate.

11. **Live system.** The final demo runs the full path from live camera to display. A prerecorded image passed through the host model is not the demo.

Required Demonstrations

1. **Bandwidth budget.** Run the live stream and change the allowed payload budget during the presentation. Show the reconstruction and frame rate as fewer or more channels are sent. The budget may be imposed manually.

2. **Progressive channels.** Encode one live frame once, then show the reconstruction from $1, 2, \ldots, N$ channels as a grid, slider or animation. Do not encode the frame N times.

3. **Decoder capacity.** Take one frame and show how the pairs trained with $D_1, \ldots, D_4$ reconstruct it across several values of k (for example $k = 1, 3, 6, 12$), as a grid of decoders against prefixes. The encoder architecture and payload size are identical in every row. This should make the asymmetry result visible.

Expected Milestones

Week 1

- Baseline codec trained, with ordered channels verified.
- Kodak evaluation pipeline and JPEG comparison in place.
- Model-focused evaluations completed.
- Camera capture working and BLE throughput measured.
- Memory wall identified.
- Encoder exploration done, including the different encoder idea.
- Final encoder architecture fixed and running on the Nano.
- CNN decoder-capacity ablation trained.

Week 2

- Full evaluation of all Week 1 models: quality and device metrics over every required k.
- Final encoder–decoder configuration selected and trained properly.
- Alternative decoder architecture trained and evaluated on the same latent.

Week 3

- QAT of the final model, with the pre- and post-quantization comparison.
- Full camera–encoder–BLE pipeline running on the Nano with latent-prefix bitrate control.
- Three live demonstrations working.
- Final evaluation finished.

Tools and Measurement

Measure latency on the Nano with the DWT cycle counter or `micros()`. Measure peak SRAM at runtime where possible, and do not report only the theoretical sum of tensor sizes if the runtime high-water mark differs. Measure BLE throughput in successfully delivered application payload bytes per second.

Record enough to reproduce every result: training dataset, preprocessing, loss, architecture, latent shape, quantization configuration, runtime version, Arduino core, tensor-arena size, camera configuration, BLE settings and code commit.

References

[1] A. Hojjat, J. Haberer and O. Landsiedel, "MCUCoder: Adaptive Bitrate Learned Video Compression for IoT Devices," Workshop on Machine Learning and Compression, NeurIPS 2024. arXiv:2411.19442. Code: https://github.com/ds-kiel/MCUCoder

[2] J. Ballé, V. Laparra and E. P. Simoncelli, "End-to-End Optimized Image Compression," ICLR 2017. arXiv:1611.01704.

[3] Z. Wang, E. P. Simoncelli and A. C. Bovik, "Multiscale Structural Similarity for Image Quality Assessment," Asilomar Conference on Signals, Systems and Computers, 2003.

[4] R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," CVPR 2018. arXiv:1801.03924. Code: https://github.com/richzhang/PerceptualSimilarity

[5] T.-Y. Lin *et al.*, "Microsoft COCO: Common Objects in Context," ECCV 2014. arXiv:1405.0312.

[6] Eastman Kodak Company, "Kodak Lossless True Color Image Suite."

[7] L. Lai, N. Suda and V. Chandra, "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs," arXiv:1801.06601, 2018.

3.3 Project B3 — Cascaded Keyword Spotting for Real-Time Robot Control (Option 2)

Supervisor: Muhammad Ahmad

Overview

Build an always-listening voice command system on the Arduino Nano 33 BLE Sense that drives a simple two-dimensional robot simulator on a host PC over BLE. The user says a wake word followed by an isolated command, as in "<wake word>, left".

The scope is **streaming wake-word detection followed by an isolated command**. Keyword extraction from natural running speech, which needs phone-level or CTC acoustic modelling, is out of scope. So is push-to-talk: the device listens continuously.

The constraint that shapes the whole project is arithmetic you will derive yourself from your baseline. A one-second window at a 250 ms hop requires four inferences per second, so if feature extraction and inference together cost more than 250 ms, the always-on path cannot keep up with real time at all. The **CPU duty cycle**, the fraction of each hop period the processor is busy, is therefore the primary system metric of this project: it decides whether always-on listening is possible, and how much time is left for the radio, the application, and idling.

Starting Point

- **Platform:** `tensorflow/tflite-micro-arduino-examples`, example `micro_speech`, which runs unmodified on this board with the built-in microphone. It establishes that the audio path, the runtime, and the build work. Its front end is the default for both training and deployment.
- **Task and data:** Speech Commands V2 [6] with its official speaker-disjoint splits. Use an eleven-class label set: a wake word, the eight robot commands (*left*, *right*, *up*, *down*, *go*, *stop*, *on*, *off*), *unknown*, and *silence*. The wake word is your choice among the dataset's multi-syllabic words outside the command set; *Marvin*, used by the MLPerf Tiny v1.3 streaming benchmark [3] because it has the most samples of such words, is one example.
- **Streaming protocol:** Evaluate on a long continuous noisy stream. If you choose *Marvin*, the v1.3 benchmark's 20-minute stream (50 wake-word instances mixed with music, speech, and traffic noise) and its quality target of at most eight false positives and eight false negatives serve as the reference. For another wake word, synthesize an equivalent stream by the same recipe. The quality target is a reference point, not a pass criterion; progress is judged against your own baseline.
- **Tooling:** `mlcommons/tiny`, `benchmark/training/keyword_spotting` [2], for its conversion path only: INT8 quantization against a representative calibration set, emission of a TFLM model, and conversion to a C array. Reuse that plumbing, but not its model.
- **Evaluation path:** How test clips and the stream reach your model is part of the work. Results may come from the device itself, or from the host running the identical INT8 model and front end, provided a device spot-check confirms host–device agreement. Document the path you chose.

Scope and Tasks

1. **Baseline and real-time deficit:** Train a deliberately plain convolutional network on the front-end features: a few standard 2D convolutions, pooling, and one dense layer, with no depthwise-separable blocks, attention, or architecture search. Quantize it to INT8, deploy it, and establish:
   - effective INT8 throughput in MACs/s, runtime flash overhead, and arena overhead above the theoretical activation footprint;
   - the operator profile, with CMSIS-NN and reference-kernel dispatch identified per operator, and feature extraction timed separately;
   - clip-level accuracy on the official test split, on host and on device;
   - parameter and MAC counts by layer (the two are rarely concentrated in the same place);
   - the real-time deficit: your window and hop geometry against measured time, expressed as a CPU duty cycle, and whether the baseline can run always-on at all.
2. **Limitations of the baseline:** Evaluate the unchanged baseline under the streaming protocol. Report the gap between clip accuracy and streaming performance, and attribute it by testing candidate causes rather than asserting them: keyword misalignment within the window, negatives drawn from isolated words rather than continuous speech, and the absence of posterior smoothing, so that one utterance fires repeatedly across overlapping windows.
3. **Initial Processing (required):** Act on what the previous task measured. Expect to need some of: randomized keyword placement across the window during training rather than the standard small time shift; partial-word windows labelled as negatives; continuous-speech negatives, from the background audio shipped with Speech Commands and a small slice of a read-speech corpus such as LibriSpeech [10]; and posterior smoothing with a refractory period. The refractory period adds latency and must appear in the end-to-end budget.
4. **Architecture (at least one required):** Replace the baseline with one of the two established directions, and justify the choice with a clear rationale grounded in your baseline profile or a small experiment (for example, host accuracy against MACs for one candidate of each family, or on-device latency of a single block of each). Report the delta under the same streaming protocol.
   - *Depthwise-separable CNNs*: This is where the field settled for this task and budget [1], and what the MLPerf Tiny reference adopts [2]. Latency-aware search over this family [7] shows how to choose between candidates; MAC count is a poor proxy for latency on this hardware, and your operator profile will show why.
   - *Fully attentional models* [4]: A reduced attention model of 1,646 parameters with two output classes took 26 million cycles per inference, with softmax and layer normalization kept in floating point because quantizing them cost too much accuracy [5]. That figure was measured on a RISC-V core, not a Cortex-M4, but at 64 MHz it would be roughly 400 ms. Whether attention can be

made competitive at this clock rate is open, and a well-measured negative result is a legitimate outcome.

Optionally, implement the other family as well and compare the two on the same harness under matched memory budgets. An early-exit branch in the command classifier with a tuned confidence gate may also be added; a single intermediate classifier has cut average operations by 20–38% for at most 1% accuracy loss on this class of workload [8].

5. **Fit the compute budget (at least one required).** Report CPU duty cycle before and after.

   - *Hard-routed cascade*: an energy or voice-activity gate, a small wake detector, and the command classifier invoked only on wake, with one path executing per window. Report the operating curve relating wake threshold to duty cycle, missed commands, and false accepts per hour.

   - *Incremental streaming computation*: cache state and compute only the new hop rather than the full window, using causal depthwise-separable convolutions or SVDF, and apply the same reasoning to the front end by computing only new feature frames and sliding the buffer.

   Optionally, implement the other strategy and compare on the same harness under matched memory budgets. Duty cycle can also be reduced below the model: runtime work beyond the TFLM defaults (eliminating reference-kernel fallbacks, tuning the arena, or folding feature normalization into quantization parameters [9]), reported per operator; or your own MFCC or log-mel front end in place of the `micro_speech` one, with host–device feature parity demonstrated.

6. **Compression (required):** Quantization-aware training, plus at least one of structured channel pruning, knowledge distillation from a host-side teacher, or low-rank factorization of the dense layers. The baseline is already INT8, so report deltas against post-training INT8, not against FP32. Report the benefit and trade-offs of each technique clearly, on both streaming quality and duty cycle, SRAM, and flash.

7. **Integration:** A BLE command channel to a 2D simulator on the host (existing simple simulators are fine), demonstrated live on the target hardware. Measure end-to-end latency from the end of the utterance to the simulator's state change, decomposed into capture, features, smoothing delay, inference, radio, and host render. Optionally, record an in-domain set through the board's microphone in the demo room and report the accuracy drop from benchmark data.

Metrics for every change: clip accuracy (host and device), streaming false positives and false negatives, command accuracy after wake, latency per pipeline stage, per-operator latency, CPU duty cycle, peak SRAM (tensor arena plus audio and feature buffers), and flash.

Fallbacks

If the compute-budget step is not converging by the second milestone, reduce the cascade to two stages and spend the time on compression. If the attention route cannot fit the hop period after reasonable effort, report the measured negative result and complete the

architecture task with a depthwise-separable CNN. If BLE is unreliable, use USB serial for the simulator link and report the substitution.

Expected Outcomes

- A working always-on voice command system with a measured CPU duty-cycle improvement over your own baseline, and a clear account of each change or of why further improvement was not achievable.
- The full reporting table with all metrics per change, Pareto plots, and, for a cascade, the operating-point analysis relating threshold to false accepts per hour and duty cycle.
- A reproducible build in a well-maintained codebase (toolchain and core versions, build flags, arena size, operator set) and a live demo on the target hardware.

Expected Milestones

1. **By first presentation (Week 1):** `micro_speech` running; baseline trained, deployed, and characterized, including the real-time deficit; evaluation path working and the streaming failure attributed.
2. **By second presentation (Week 2):** Streaming fixes, the architecture choice, and the compute-budget step.
3. **By final presentation (Week 3):** Compression, integration, and live demo.

Tools and Measurement

Time each stage on the device with the DWT cycle counter or `micros()`, and compute CPU duty cycle as busy time per hop divided by the hop period. Read peak arena usage from the interpreter (`arena_used_bytes()`) and flash and static SRAM from the build output. Power is not measured in this project; duty cycle is its proxy. Record the Arduino core, TFLM library, and CMSIS-NN versions with every result.

Requirements

- **Baseline:** the plain INT8 CNN from Task 1, deployed on the board and fully characterized, including the real-time deficit, with its streaming failure attributed by experiment (Task 2) before any optimization.
- **Pipeline:** a reproducible build, a documented evaluation path (on device, or on host with a device spot-check), the streaming test set, and a harness that logs every per-change metric; streaming fixes (Task 3) in place.
- **Architecture and compute budget:** one architecture family with a justified choice (Task 4), and one compute-budget strategy with duty cycle reported before and after (Task 5).
- **Compression:** quantization-aware training plus at least one further technique, reported against post-training INT8 (Task 6).
- **Improvement lineage:** each change is applied on top of the last accepted configuration, forming one documented chain from baseline to final system. Every step is

measured with the full metric set on the same test split and stream and marked as kept or discarded, with the reason. Discarded changes remain in the lineage.

- **Optional extensions:** at most one is expected; completing the required tasks with a clear narrative carries more credit.

References

[1] Y. Zhang, N. Suda, L. Lai and V. Chandra, "Hello Edge: Keyword Spotting on Microcontrollers," arXiv:1711.07128, 2017.

[2] C. Banbury *et al.*, "MLPerf Tiny Benchmark," NeurIPS Datasets and Benchmarks Track, 2021. arXiv:2106.07597. Code: https://github.com/mlcommons/tiny

[3] MLCommons, "A New TinyML Streaming Benchmark for MLPerf Tiny v1.3," 2025. https://mlcommons.org/2025/09/mlperf-tiny-v1-3-tech/

[4] A. Berg, M. O'Connor and M. T. Cruz, "Keyword Transformer: A Self-Attention Model for Keyword Spotting," Interspeech 2021. arXiv:2104.00769.

[5] "KWT-Tiny: RISC-V Accelerated, Embedded Keyword Spotting Transformer," arXiv:2407.16026, 2024.

[6] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," arXiv:1804.03209, 2018.

[7] C. Banbury *et al.*, "MicroNets: Neural Network Architectures for Deploying TinyML Applications on Commodity Microcontrollers," MLSys 2021. arXiv:2010.11267.

[8] N. P. Ghanathe and S. J. E. Wilton, "T-RecX: Tiny-Resource Efficient Convolutional Neural Networks with Early-Exit," 2022.

[9] L. Lai, N. Suda and V. Chandra, "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs," arXiv:1801.06601, 2018.

[10] V. Panayotov, G. Chen, D. Povey and S. Khudanpur, "LibriSpeech: An ASR Corpus Based on Public Domain Audio Books," ICASSP 2015.

4 Expectations and Scope of Work

4.1 Scope of Work

Regardless of the track chosen, students are expected to follow these key steps:

1. **Problem Formulation:** Define a clear problem statement, research objectives, and underlying hypotheses. Establish the context of the problem in the domain of edge AI, model compression, compilation, or distributed learning.
2. **Literature Review:** Conduct a detailed study of existing methods and relevant research to identify the current state of the art. Highlight the limitations of existing approaches and justify the proposed methodology.
3. **Dataset Processing and Feature Engineering:** Prepare and preprocess datasets to ensure data consistency and quality. Apply suitable feature extraction or transformation techniques to enhance model performance and generalization.
4. **Model / System Implementation:** Design, implement, and evaluate the models, transformations, or toolchain components aligned with the chosen project. Ensure reproducibility through clear documentation and version control.
5. **Hardware Deployment:** Deploy on the target device and report against its constraints. Document the toolchain, runtime, build flags, and any device-specific configuration required to reproduce your results.
6. **Performance Evaluation:** Compare variants using quantitative metrics such as accuracy, latency, throughput, power consumption, and memory footprint. Perform trade-off analysis to interpret results in light of system constraints.
7. **Final Report and Presentation:** Prepare a comprehensive technical report that summarizes the objectives, methods, and experimental findings.

4.2 Assessment

The project will be assessed based on the following components:

- **Project Report, Code & Documentation (10 %)**: Detailed report covering problem formulation, methodology, implementation, and results. Well-structured and documented codebase.
- **Final Presentation and Viva (10%)**: Summary of the project findings highlighting key contributions in the final presentation.
- **Weekly Progress Presentations (5 % + 5 %)**: Weekly presentations describing the progress and milestones achieved in-class.

We encourage you to use a template from your favorite machine learning conference (e.g., NeurIPS or ICML) for the report. We encourage Masters and PhD students to use the LaTeX template for their reports. You can seek guidance from TAs regarding the sections that are required to be included in the report.

5 Timeline of Deliverables

We want you to adhere to the following timelines.

- **Deliverable 1:** Project selection and group formation.
  - Due: October 18, 2026
  - The spreadsheet for groups can be found at this link.
  - Since every project has fixed capacity, projects will be assigned on a first-come, first-served basis (decided by the entry log on the spreadsheet).
- **Deliverable 2:** Weekly Progress Presentations
  - Presentation 1: Week 13 November 29, 2026
  - Presentation 2: Week 14 December 06, 2026

  Any group member may be asked to present.
- **Deliverable 3:** Final Presentation, Demo and Viva
  - Due: December 20, 2026.
- **Deliverable 4:** Submit code (documented) and final report
  - Due: December 24, 2026
  - The report is expected to include sections covering:
    - Abstract
    - Introduction
    - Literature Review
    - Detailed Project Overview
    - Problem Formulation
    - Methodology
    - Results
    - Discussion
    - Conclusion